ACS-2947-002/003
Assignment 2
Due by Thursday, February 27, 11:59 pm

Instructions

- Submit your `.java` files (together in an `Assign2.zip` file) via Nexus.
- Include your name and student number as a comment in every file.
- Document the classes using Javadoc notation.
- Include comments as needed and use exception handling where necessary.

PART A (30 marks)

In the classic version of the game Simon, the device creates a series of tones and lights and requires a user to repeat the sequence. If the user succeeds, the series becomes progressively longer and more complex. Once the user fails, the game is over.

A simple text-based simulation of the game will display the words *red, blue, green,* or *yellow*. Each word will be displayed briefly before displaying the next word. The first level starts with one colour. If the user repeats the sequence successfully then the next level displays two colours. If the user repeats the sequence successfully then the next level displays 3 colours. This will continue until the user fails to produce the correct sequence.

1. Create the generic `ArrayList` class that implements the provided `List` interface (note that `List` extends `Iterable`)
   a. Overload the `add` method: include another `add` method that will have one parameter, an element that adds to the ***end*** of the list.
   b. Make your `ArrayList` dynamic: the array should grow to double its current capacity if it runs out of space and shrink to half its current capacity when the number of elements in the arraylist goes below *N*/4, where N is the current capacity of the array. Modify `add()` and `remove()` methods and include a `resize()` method to support the dynamic structure. Set the default capacity to 4.
   c. **Override** the `equals` method of the Object class so that it checks if the `ArrayList` is equivalent to the given instance. Consider an appropriate definition of equivalence.

2. Create a `PartA_Driver` class to simulate the game of Simon (text-based version). Your program will use two `ArrayList` instances to store and compare the program's growing colour sequence and the user's input.

   a. Use the standard output with `Scanner` class or `JOptionPane` for input
      - Each *next* word/colour should be randomly chosen
      - You will need to briefly display and then clear each word. You may use `Thread.sleep()` with a series of `"\b"` to remove the characters or a series of `"\n"` to clear the screen, etc.

- Any input is considered valid: incorrect input means an unmatched sequence and the end of the game. Words in the input are separated by spaces.

b. Display the user's final score when the game is over.

Note:

- Test that your code for the dynamic implementation of the ArrayList works. This will be evaluated but you do not have to include the driver code for this in your submission. Also test that all other methods work.
- Enums (e.g., `Colour`) are optional
- Do not use Java's ArrayList for this task

Sample output:

```
System:     Let's play Simon!

System:     green            (displays briefly then clears)
User:       green
System:     Correct! Your current score is 1

System:     yellow           (displays briefly then clears)
            green            (displays briefly then clears)
User:       yellow green
System:     Correct! Your current score is 2

System:     red              (displays briefly then clears)
            yellow           (displays briefly then clears)
            green            (displays briefly then clears)
User:       red yellow green
System:     Correct! Your current score is 3

System:     green            (displays briefly then clears)
            green            (displays briefly then clears)
            red              (displays briefly then clears)
            blue             (displays briefly then clears)
User:       green blue red blue
System:     Game over! Your score is 3.
```

PART B (50 marks)

Implement a Positional List using an **array.** Refer to page 281 in your textbook or see the Appendix.

1. Create two classes called `ArrayPositionalList` (APL) with a nested class `Location` that implements the provided `PositionalList` and `Position` interfaces, respectively.

2. Demonstrate the use of your APL in a `PartB_Driver` class by doing the following. Create a static method called `removeConsecutiveDuplicates` that removes any consecutive duplicate strings from an array positional list of Strings and returns the number of strings left after the removal. After calling the method, the positional list parameter should contain the same sequence of strings as before but with any consecutive duplicates removed. Illustrate your method using the following sets of strings and display the content of the list after executing the method.

    a. harry will tom tom tom charles
    b. harry harry tom will meg harry
    c. tom will harry charles meg
    d. meg meg tom tom tom tom harry
    e. meg meg tom james charles charles james harry harry harry

NOTE: You **MUST** use a combination/variation of ALL the add and set methods (`AddFirst, AddLast, AddBefore, AddAfter,` and `set`) in creating **one** of the original lists above. All the sample lists above must be created and illustrated in your driver class.

You **MUST NOT** use any other auxiliary data structure e.g., arrays in your implementation of `removeConsecutiveDuplicates`

3. Create a static method called `InsertionSort` that sorts the items of an array positional list of characters based on the insertion sort algorithm. Refer to slides 16 – 20 of L02_ for the array-based algorithm and implementation. Illustrate your sorting method with the following list of characters:
   A, M, A, Z, I, N, G, R, A, C, E

To get you started, a beginning `ArrayPositionalList` implementation with the nested class `Location` is provided.

- `Location` implements the `Position` interface (just like `Node` in a *linked* positional list). Note that there is no `next` or `prev`, but only an integer `index` and generic `element`.
- Fields and constructors are included, array stores `Location` objects.

a) Add your `size()` and `isEmpty()` methods.
b) Implement the `first()` and `last()` methods: how would you get the first and last elements from the array? This should form a basis of how to move from linked to array. From there you can start thinking about how to convert all the methods from linked-based to array-based implementation.

Consideration:

With `LinkedPositionalList`, you get the previous and next positions through the node (which is a `Position`) and `getNext()` and `getPrev()` methods. With `ArrayPositionalList` you get the next and previous through the `Position` as well, but with the `getIndex()` method and the array.

Instead of simply calling `node.next()` you will find out what `Location.getIndex()` is, then return the `Location` at the *next index* of your array.

Note:

- You will need a way to validate and explicitly cast `Position` objects to `Location` objects in order to use `Location` methods like `getIndex()`
- The `index` field of the `Location` class needs to be updated with any methods that require a *shift* in elements.
- Override the `toString` method to display both the index and element. For example, the content of the `ArrayPositionalLists` for the last set of strings above will be displayed as in the sample output after executing `removeConsecutiveDuplicates`. This can be useful for testing/debugging.
- The APL is not required to be dynamic. It is also not required to be Iterable.

Suggestions:

- Many methods declare exceptions in their signature: most can be handled in common private utility methods
- Think about how you can reuse code e.g., for the different add methods
- Test each method as you write it.

Sample Output

The sample shows only the last set of Strings for Question 3, you should illustrate all the provided string lists, not only this one.

```
Original positional list:
[0] meg [1] meg [2] tom [3] james [4] charles [5] charles
[6] james [7] harry [8] harry [9] harry

Number of entries after call: 6
List with duplicates removed:
[0] meg [1] tom [2] james [3] charles [4] james [5] harry

...

Sorted characters using Insertion sort algorithm:
[0] A [1] A [2] A [3] C [4] E [5] G [6] I [7] M [8] N [9] R [10] Z
```

Submission

Submit your **`Assign2.zip`** file that includes all the assignment files (`List.java`, `ArrayList.java`, `PartA_Driver.java`, `Position.java`, `PositionalList.java`, `ArrayPositionalList.java`, `PartB_Driver.java`, any other class used in your code) via **Nexus**.

Appendix

Implementing a Positional List with an Array

We can implement a positional list L using an array A for storage, but some care is necessary in designing objects that will serve as positions. At first glance, it would seem that a position p need only store the index i at which its associated element is stored within the array. We can then implement method getElement(p) simply by returning $A[i]$. The problem with this approach is that the index of an element e changes when other insertions or deletions occur before it. If we have already returned a position p associated with element e that stores an outdated index i to a user, the wrong array cell would be accessed when the position was used. (Remember that positions in a positional list should always be defined relative to their neighboring positions, not their indices.)

Hence, if we are going to implement a positional list with an array, we need a different approach. We recommend the following representation. Instead of storing the elements of L directly in array A, we store a new kind of position object in each cell of A. A position p stores the element e as well as the current index i of that element within the list. Such a data structure is illustrated in Figure 7.8.

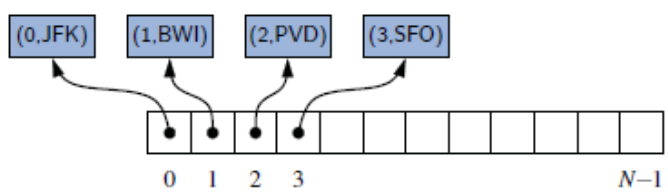

Figure 7.8: An array-based representation of a positional list.

With this representation, we can determine the index currently associated with a position, and we can determine the position currently associated with a specific index. We can therefore implement an accessor, such as before(p), by finding the index of the given position and using the array to find the neighboring position.

When an element is inserted or deleted somewhere in the list, we can loop through the array to update the index variable stored in all later positions in the list that are shifted during the update.

Efficiency Trade-Offs with an Array-Based Sequence

In this array implementation of a sequence, the addFirst, addBefore, addAfter, and remove methods take $O(n)$ time, because we have to shift position objects to make room for the new position or to fill in the hole created by the removal of the old position (just as in the insert and remove methods based on index). All the other position-based methods take $O(1)$ time.